

GENERATIVE AI FOR SOFTWARE ENGINEERING

CAPSTONE PROJECT

EXECUTIVE SUMMARY

PROJECT CORPUS FORGE

EXECUTIVE SUMMARY

1. PROJECT OVERVIEW

This project challenges student teams to design, build, and critically evaluate an AI-powered software platform that transforms heterogeneous document collections into actionable knowledge.

The objective is not simply to build an application, but to demonstrate engineering judgment, experimentation, technical ownership, and effective collaboration with AI-assisted development tools.

Students must design a system capable of:

- Ingesting multiple document types
- Organizing and managing document collections
- Retrieving relevant information from selected corpora
- Generating useful knowledge artifacts
- Allowing users to steer AI behavior through prompts
- Persisting documents and generated outputs
- Monitoring AI usage and cost

The project emphasizes experimentation, iteration, and evidence-based engineering decisions.

2. CORE PLATFORM REQUIREMENTS (LAYER 1)

Each team must implement a working web application that supports the following capabilities:

CORPUS INGESTION

The platform must process at least three document types, including:

- Text or Markdown
- PDF
- Source code files

CORPUS MANAGEMENT

Users must be able to:

- Add documents
- Remove documents
- Browse collections
- Select active documents for AI interaction

2. CORE PLATFORM REQUIREMENTS (LAYER 1)

RETRIEVAL-GROUNDED AI WORKFLOWS

The platform must support:

- Chat-based exploration and question answering
- Flashcard generation
- Quiz generation
- Source-code analysis reports
- Code review
- Architecture and control-flow reports

PROMPT STEERING

Users must be able to customize generation through:

- Audience level
- Output format
- Tone
- Creativity
- Task instructions

2. CORE PLATFORM REQUIREMENTS (LAYER 1)

PERSISTENCE

Documents, metadata, and generated artifacts must persist across sessions.

COST OBSERVABILITY

The platform must expose at minimum:

- Request count
- Token usage

3. ENGINEERING CHALLENGES (LAYER 2)

Each team must complete two advanced engineering challenges.

Available challenge areas include:

1. Retrieval Experimentation

- Compare multiple retrieval strategies and evaluate trade-offs.

2. Prompt Engineering

- Iteratively improve prompts for quality, reliability, and reduced hallucination.

3. Visualization

- Create interactive visualizations derived from corpus data.

4. Reliability and Testing

- Design tests for both normal and failure scenarios.
- Students must document:
 - What worked
 - What failed
 - What required redesign

4. DELIVERABLES

Each team must submit:

1. Working Application: A runnable local web application.
2. GitHub Repository
 - Source code
 - README.md
 - REPORT.md
 - Prompt history and journals
 - Generated artifacts
 - Presentation slides
3. Technical Report
 - Team members
 - Initial architecture
 - Engineering decisions
 - Division of work
 - AI collaboration
 - Failures and redesigns
 - Lessons learned

5. PRESENTATION

Each team delivers:

- 3-minute live demo
- 12-minute technical presentation
- 5-minute Q&A session

Topics should include:

- Motivation
- Architecture
- Engineering challenges
- AI collaboration
- Lessons learned

7. KEY MESSAGE

This project rewards engineering maturity over feature count.

Higher marks are awarded to teams that:

- Learn unfamiliar concepts quickly
- Use AI as an engineering partner rather than a shortcut
- Justify technical decisions with evidence
- Document failures honestly
- Demonstrate full ownership of their system

The goal is not merely to build software.

The goal is to prove the ability to learn, architect, validate, and critically evaluate AI-assisted engineering decisions.